

Topic: IronPredict — Analysis and Prediction of Ironman 70.3 Performance

Context

An Ironman 70.3 triathlon strings together a 1.9 km swim, a 90 km bike, and a 21.1 km run, separated by two transitions. Final performance does not depend on a single discipline but on how the effort is managed across the whole race. A pacing error — typically going too hard on the bike — often triggers a collapse (a "blow-up") on the run. This project aims to build an analytics tool capable of identifying the athlete's limiting factor, determining where to invest training time for the best gain in finish time, and detecting the risk of a blow-up before it happens.

Problem Statement

What limits my Ironman 70.3 performance, where should I invest my training time for the best return on finish time, and can the risk of a race-day blow-up be detected before it occurs?

Objectives

- Clean and structure race and training data (FIT / TCX / GPX).
- Diagnose the limiting factor: the weakest discipline or transition relative to the age group.
- Estimate the return on investment of each discipline (seconds gained per unit of improvement).
- Define and model the blow-up risk.
- Build a performance and risk dashboard.

Features

- Import activity files (FIT / TCX / GPX) and race results.
- Extract metrics: normalized power, pace, HR, cadence, elevation, splits.
- Compute HR/pace decoupling (aerobic drift) and the variability index.
- Compare splits against age-group distributions (percentiles, z-scores).
- Classify blow-up risk: at-risk vs. controlled race.
- Decoupling monitor showing the moment the risk becomes detectable.
- Dashboard: diagnostic, training ROI, and risk.

Technical Pipeline

1. Collect race files and training logs.
2. Cleaning: handle GPS dropouts, pauses, resampling.
3. Feature engineering: normalized power, VI, HR/pace decoupling, bike→run ratio, first/second-half split ratio.
4. Exploratory analysis: pace / power / HR curves, time distribution by discipline.
5. Benchmarking by percentiles against the age group.
6. Regression: finish time as a function of splits and training load (CTL/ATL/TSB).
7. Classification: Logistic Regression / Random Forest for blow-up risk.
8. Store results in a SQL database.
9. Visualization in Power BI or Tableau.

Tools Used

Python, Pandas, fitparse / python-tcxparser, Scikit-learn, Machine Learning, SQL, BI, Data Visualization.

Possible Datasets

- Your own FIT / TCX files exported from Garmin or Strava (race and training).
- Public Ironman 70.3 results (age-group splits, e.g. the Coach Cox dataset).
- Training-load data (TrainingPeaks: CTL, ATL, TSB) if available.
- Environmental data (temperature, wind, water temperature) to compare multiple races.

Defining the Blow-Up

The target variable must be measurable. The chosen definition: HR/pace decoupling above 5% on the run, a sign of cardiac drift and an impending collapse. Alternatives: a positive split (second half markedly slower than the first) or a large gap between predicted and actual finish time.

Expected Deliverables

- Python notebook (cleaning, features, analysis).
- SQL database.
- ML model (regression + classification).
- BI dashboard (diagnostic, ROI, risk).
- Final report (LaTeX).

Methodological Note

The predictive part (blow-up) requires several races or long sessions to have labeled examples: a model built on a single race is descriptive, not predictive. With few races, keep blow-up detection descriptive (the decoupling monitor) and lean the modeling weight onto the training-load data.

Level

Advanced.